

EVA Project — Consolidated Downloadable PDF

Compiled from the EVA architecture, implementation, security, memory, tool, deployment, testing, and handoff documents.

EVA v0.1 — Master Engineering Specification

EVA is a modular intelligence framework designed to assist humans through mission understanding, research, planning, knowledge organization, controlled automation, and transparent decision support.

Core components:

- EVA Core — central coordinator
- Pixie — human interaction layer
- Nova — research and creative intelligence
- Sentinel — trust and security layer
- Ghost — memory and continuity

Core operating model:

Mission received → understand objective → create task plan → security review → execute tasks → verify results → store knowledge → report completion.

Recommended stack:

Python, FastAPI, PostgreSQL, vector database/knowledge graph, Docker, Git.

Build milestones:

1. Skeleton
2. Intelligence
3. Memory
4. Security
5. Deployment

EVA v0.1 — Complete System Integration Plan

The complete system connects User → Pixie → EVA Core → Nova/Sentinel/Ghost → Tool Management → Knowledge Foundation.

Mission flow:

User request → Pixie interprets intent → EVA creates mission → Sentinel checks authorization → EVA creates task plan → agents receive tasks → tools are used → results are verified → Ghost stores approved memory → EVA creates final response → Pixie presents result.

System boundaries:

Allowed: research, analysis, organization, planning, document creation, user assistance.

Restricted: external communication, file modification, system changes, deployment actions require approval.

Prohibited: removing safety controls, hidden operation, unauthorized access, false claims of certainty.

EVA v0.2 — Agent Implementation Framework

Every agent has a specific mission, limited responsibilities, defined permissions, clear inputs/outputs, and traceable history.

Universal structure:

Identity, Purpose, Permissions, Memory Access, Available Tools, Processing Logic, Reporting System, Audit Trail.

Agents:

- EVA Core — coordinator
- Pixie — human interface
- Nova — research and exploration
- Sentinel — authorization, risk, security
- Ghost — memory keeper

Agent lifecycle:

Created → Registered → Authorized → Active → Monitored → Updated → Retired.

EVA v0.2 — Database Implementation & Memory Engine

Ghost answers:

What do we know? Where did we learn it? How confident are we? Should we trust it?

Memory levels:

Short Memory — current task

Long Memory — approved information

Knowledge Memory — research data

Primary PostgreSQL tables:

users, missions, tasks, agents, memory_records, memory_sources, permissions, audit_events, system_versions.

Memory classifications:

FACT, CLAIM, THEORY, IDEA, DECISION.

Ingestion:

Extract → identify source → classify → assign confidence → check conflicts → store → create relationships.

Memory must preserve history, sources, changes, and conflicting information.

EVA v0.2 — Tool Integration Framework

All external capabilities pass through a controlled Tool Gateway.

Flow:

Agent request → Tool Gateway → Sentinel review → Tool execution → result validation → memory update.

Tool categories:

- Information — search, documents, knowledge databases
- Processing — text analysis, data processing
- Storage — databases, files, archives
- Communication — messaging and notifications
- Development — code analysis, testing, build tools

Tools cannot change their own permissions, bypass Sentinel, modify audit logs, hide results, or expand access automatically.

EVA v0.1 — Alpha Implementation Roadmap

Development sequence:

Foundation → Memory → Security → Agents → Tools → Automation → Advanced Intelligence.

Phases:

1. Foundation Core
2. Mission Engine
3. Sentinel Security
4. Ghost Memory
5. Pixie Interface
6. Nova Research
7. Agent Coordination
8. Testing Environment
9. First Demonstration
10. Expansion

The first demonstration should receive a mission, create tasks, check authorization, research, store findings, and generate a report.

EVA v0.1 — Alpha Prototype Code Specification

Primary software objects:

EVA, Mission, Agent, Task, Memory, Permission, Audit Event.

Core flow:

Mission → planning → agent assignment → tool use → result evaluation → memory storage → report.

Initial API endpoints:

POST /mission/create

GET /mission/status

POST /task/create

POST /authorize

POST /memory/store

GET /memory/search

GET /audit/history

All important activity receives an audit event containing timestamp, agent, action, reason, and result.

EVA v0.1 — Developer Build Guide

First target: EVA Research Node — a controlled intelligence assistant capable of receiving missions, breaking them into tasks, managing specialized agents, researching/analyzing information, storing verified knowledge, producing reports, and maintaining an audit trail.

Recommended stack:

Python 3.12+, FastAPI, PostgreSQL, pgvector, optional knowledge graph, Docker Compose.

Build order:

EVA Core → Mission System → Agents → Memory → Security → Tools.

The first successful conversation should demonstrate Pixie intake, EVA planning, Sentinel authorization, Nova research, Ghost storage, and final EVA reporting.

EVA v0.1 — Deployment Blueprint

Deployment philosophy:

Security → Reliability → Maintainability → Scalability → Advanced Capability.

Deployment models:

- A. Local EVA Node
- B. Private Cloud EVA
- C. Hybrid EVA

Prototype hardware:

Modern computer, 16+ GB RAM, SSD, internet connection.

Recommended container separation:

EVA Core, Pixie, Nova, Sentinel, Ghost, Database, Monitoring.

Production requirements:

Encrypted storage, authentication, permission control, audit logs, backups, version control, recovery process.

EVA v0.1 — Testing & Validation Framework

EVA is successful when it understands missions correctly, uses tools responsibly, preserves accurate memory, explains decisions, recovers from failures, and respects authorization boundaries.

Testing layers:

Functional, Security, Intelligence, Memory, Behavior, Recovery, Human Acceptance.

Test areas:

- Mission creation and routing
- Pixie communication
- Nova research
- Ghost memory accuracy
- Sentinel authorization
- Prompt injection
- Permission bypass
- Conflicting sources
- Missing data
- Agent failure

Human authority remains the final control point.

EVA v0.1 — Security & Zero Trust Framework

Sentinel verifies every action before execution.

Pipeline:

Action request → identity check → mission check → permission check → risk analysis → scope validation → approve/deny → log result.

Principle:

No trust by default; everything must be verified.

Authorization levels:

0 Public

1 User Approved

2 Explicit Confirmation

3 Restricted

4 Forbidden

EVA cannot silently rewrite core rules, security policies, permission systems, or mission priorities. Architectural changes require proposal, review, human approval, version update, and deployment.

EVA v0.3 — Engineering Handoff Package

First usable product:

A private research assistant with controlled agents.

Minimum flow:

User mission → Pixie → EVA Core → Sentinel → Nova → Ghost → final report.

Missing from the architecture package:

Runnable code, exact model choices, API contracts, database migrations, test suite, deployment scripts, and user interface.

Recommended initial stack:

Python + FastAPI, PostgreSQL, pgvector, Docker Compose, simple React interface.

The next engineering agent should build rather than rewrite the philosophy.

EVA v0.3 — Build Handoff

Implementation instruction:

Do not redesign the architecture. Build the first runnable EVA v0.3 Research Node.

Initial capabilities:

1. Accept a user mission.
2. Convert it into a structured mission.
3. Generate tasks.
4. Assign tasks to agents.
5. Run Sentinel authorization.
6. Execute a controlled Nova research task.
7. Validate/store knowledge through Ghost.
8. Produce a report with findings, sources, confidence, and limitations.
9. Record the operation in an audit trail.

Initial tables:

users, missions, tasks, agents, memory_records, sources, permissions, audit_logs.

Definition of done:

A mission can be submitted, tasks generated, authorization checked, research executed, validated memory stored, report generated, and audit history recorded.